

PART 3

30 Expert Angular Interview Questions

Part 3 - Advanced Concepts & Architecture

Real-World Scenarios & Performance Optimization

61. How do you implement custom form control with `ControlValueAccessor`?

A complete implementation requires:

- **Implement `ControlValueAccessor` interface** with four methods:
 - `writeValue(value: any)`: Set value from model to view
 - `registerOnChange(fn)`: Store callback for value changes
 - `registerOnTouched(fn)`: Store callback for touch events
 - `setDisabledState(disabled)`: Handle disabled state
- **Provide `NG_VALUE_ACCESSOR` token** in component providers
- **Call `onChange()`** when internal value changes
- **Call `onTouched()`** when component is touched/blurred

This allows your custom component to work seamlessly with Angular forms (both template-driven and reactive).

62. Explain Angular's compilation process from code to browser.

With AOT (Production):

- TypeScript compilation to JavaScript
- Template compilation to TypeScript
- Angular compiler generates component factories
- Tree-shaking removes unused code
- Bundling and minification
- Browser loads and executes compiled code

With JIT (Development):

- TypeScript compilation to JavaScript
- Browser loads Angular compiler
- Templates compiled at runtime in browser
- Components instantiated dynamically

AOT moves compilation from runtime to build time for better performance.

63. What are the different strategies for state management in Angular?

Options:

- **Component State:** @Input/@Output, simple for small apps
- **Service with BehaviorSubject:** Shared state via services, good for medium apps
- **NgRx:** Redux pattern with actions, reducers, effects. Best for large, complex apps with predictable state
- **Akita:** Simpler than NgRx, entity-based state management
- **NGXS:** Redux alternative with less boilerplate
- **Signals:** Built-in reactive state (Angular 16+)
- **RxAngular:** Reactive state management with RxJS

Choose based on app complexity, team expertise, and scalability needs.



64. Explain the concept of Facade pattern in Angular services.

Facade pattern provides a simplified interface to complex subsystems.

In Angular:

- Create a facade service that coordinates multiple services
- Components interact only with facade, not individual services
- Hides complexity of state management, API calls, business logic
- Provides clear, focused API for components

Benefits:

- Decouples components from complex logic
- Easier testing (mock single facade)
- Better maintainability
- Clear separation of concerns

65. How do you handle error globally in Angular applications?

Multiple approaches:

- **ErrorHandler:** Implement custom ErrorHandler to catch all errors globally. Override handleError() method.
- **HTTP Interceptor:** Catch HTTP errors centrally, handle 401, 403, 500 errors uniformly.
- **Router Error Events:** Subscribe to router events to catch navigation errors.
- **RxJS catchError operator:** Handle observable errors in streams.

Best practice: Combine approaches - use ErrorHandler for unhandled errors, interceptors for HTTP, and specific error handling where needed. Log errors to monitoring service (Sentry, LogRocket).